

Web Music Player

Architecture Documentation

structured after the arc42 template (v8)

Nicolà Widmer · HSLU · Web Programming Lab

2026-09-21

Contents

1	Introduction and Goals	3
1.1	Quality Goals	3
1.2	Stakeholders	3
2	Architecture Constraints	4
2.1	Technical Constraints	4
2.2	Organisational Constraints	4
2.3	Conventions	4
3	Context and Scope	5
3.1	Business Context	5
3.2	Technical Context	5
3.3	Scope Boundaries	5
4	Solution Strategy	6
5	Building Block View	8
5.1	Level 1 — Whitebox Overall System	8
5.2	Level 2 — Frontend Whitebox	8
6	Runtime View	10
6.1	Scenario: play a song from the library	10
6.2	Scenario: delete a song from a playlist view	10
6.3	Scenario: upload and parse a song	10
7	Deployment View	11
8	Cross-cutting Concepts	12
8.1	State management (frontend)	12
8.2	Component communication (frontend)	12
8.3	Authentication (frontend half)	12
8.4	Internationalisation	12
8.5	Forms	12
8.6	Styling & theming	12
8.7	Accessibility	12
8.8	Testing strategy (frontend)	12
9	Architecture Decisions	14
10	Quality Requirements	15
10.1	Quality Tree	15
10.2	Quality Scenarios	15
11	Risks and Technical Debt	16
12	Glossary	17

1 Introduction and Goals

A web-based music player. Users upload MP3 files, organise them into playlists, and play them back on a screen styled as a record player: a vinyl disc that spins while a track plays and a tone-arm that drops on play and lifts on pause. The two self-defined CRUD resources are **Songs** and **Playlists**. The full functional scope and its MoSCoW prioritisation live in `Proposal.typ`; this document describes **how** the system is built and **why**.

1.1 Quality Goals

Prio	Quality goal	Motivation
1	Turntable-like experience	The playback screen must feel like a real turntable — disc spin and tone-arm motion — while still honouring prefers-reduced-motion .
2	Smooth audio streaming & seeking	Playback starts without downloading the whole file; seeking works on mobile. Requires HTTP Range (206 Partial Content) end to end.
3	Testability & maintainability	Domain logic isolated from NestJS/PostgreSQL so a use case is unit-testable with no I/O; a clear test pyramid (Vitest / Supertest / Playwright).
4	Per-user data privacy	A user only ever sees or changes their own songs and playlists; enforced server-side, not just in the UI.
5	One-command operation	docker compose up brings the whole system up on a clean machine with no manual steps and no cloud account.

1.2 Stakeholders

Role	Expectations towards the architecture
Developer / student (Nicolà Widmer)	A structure to build within the time box; guardrails that keep decisions consistent; fast feedback from tests.
Assessor / lecturer (HSLU)	Lab requirements met (≥ 2 self-defined CRUD resources, automated tests, Docker packaging); architecture and decisions documented.
End user / self-hoster	Reliable upload and playback, a private library, a simple setup.
Future maintainer	Understandable module boundaries and ADRs that explain the “why”.

2 Architecture Constraints

2.1 Technical Constraints

Constraint	Consequence
Frontend framework: Angular	SPA, TypeScript, standalone components.
Backend: NestJS (Express platform), REST	Node.js/TypeScript HTTP layer; no GraphQL, no SSR. OpenAPI spec derived from controllers/DTOs (see ADR-0002).
Database: PostgreSQL	Relational metadata store; SQL migrations.
Audio bytes on the server filesystem (mounted volume), DB holds the path; S3-compatible storage is an optional (Could) alternative	A storage abstraction is needed so the backend is not hard-wired to fs .
Tests: Vitest (unit; Jest-compatible API, native ESM — replaces the proposal's Jest on both halves), Supertest + real PostgreSQL (integration), Playwright (E2E)	Ports must be mockable; integration tests need a real DB (and MinIO for the S3 path).
Packaging: Docker Compose , <code>docker compose up</code>	Every runtime dependency is a Compose service; config comes from env vars.
Target: modern evergreen browsers	No IE/legacy shims; <code><audio></code> + Media Session API assumed available.

2.2 Organisational Constraints

- **Single developer**, fixed academic deadline — scope is protected by the MoSCoW list in the proposal.
- Solo work — architecture must be navigable without a team to ask.

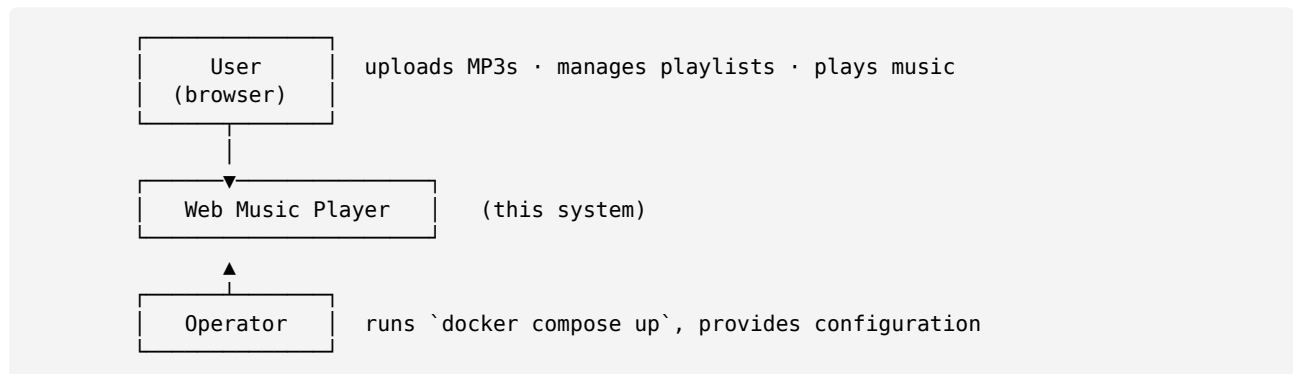
2.3 Conventions

- Backend follows **Domain-Driven Design** (tactical patterns) with a **hexagonal** (ports & adapters) layering; one module per bounded context.
- Significant decisions are recorded as **ADRs** under `Documentation/adr/` (MADR-style, numbered, immutable) and referenced from section 9.
- TypeScript everywhere; ESLint + Prettier; SQL migrations are forward-only.

3 Context and Scope

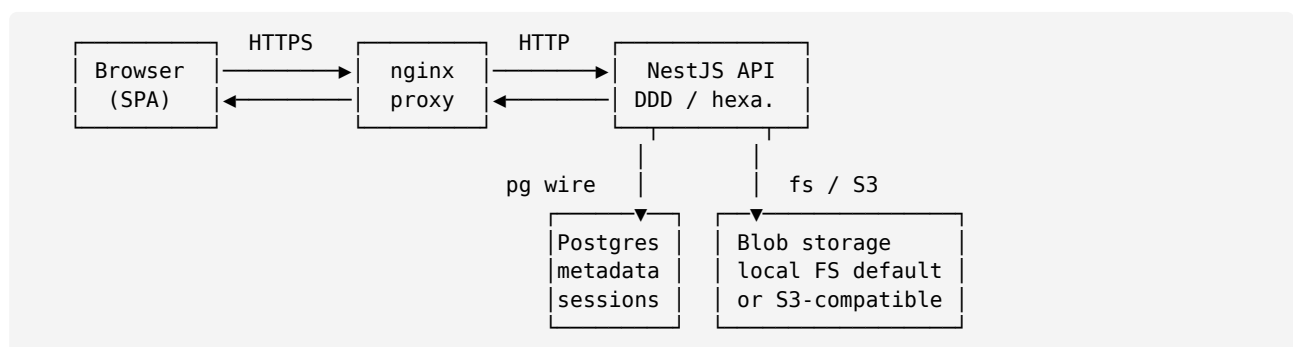
3.1 Business Context

The system is self-contained: the only external actor is the **user** in a browser. There is no third-party integration in the committed scope (no email provider — email verification and password reset are explicitly out of scope). An **operator** deploys and runs the Compose stack.



3.2 Technical Context

Channel	Protocol	Payload / purpose
Browser ↔ nginx	HTTPS	SPA assets (HTML/JS/CSS); REST/JSON API calls; audio stream with Range / 206.
nginx ↔ NestJS API	HTTP (proxy)	Reverse-proxied <code>/api/*</code> ; <code>Range</code> , <code>X-Forwarded-*</code> headers passed through.
NestJS API ↔ PostgreSQL	TCP (pg wire)	Song & playlist metadata; local user rows keyed by Keycloak sub.
NestJS API ↔ Keycloak	HTTP (OIDC)	Realm discovery & JWKS for access-token validation; Keycloak uses its own PostgreSQL (<code>keycloak-db</code>).
NestJS API ↔ Blob storage	filesystem calls <i>or</i> S3 HTTP API	Read/write/delete audio objects; range reads for streaming.



3.3 Scope Boundaries

In scope	Out of scope
MP3 upload with validation and ID3 extraction; song & playlist CRUD; record-player playback with seek / repeat / volume / resume; range streaming; session-cookie auth with per-user isolation; optional S3 storage backend.	Email verification & password reset; audio transcoding or non-MP3 formats; sharing / collaboration / social features; native mobile apps; multi-tenant administration; CDN and horizontal scaling (considered only as an evolution path).

4 Solution Strategy

Quality goal	Architectural approach	Ref
Maintainability, testability	Monorepo monolith: backend/, frontend/, Compose at the root — one deployable, one repo, simple mental model.	0001-monorepo-monolith
Testability, maintainability, one API contract	DDD + hexagonal backend on NestJS (Express platform): one feature-folder module per bounded context (songs , playlists , identity , streaming), each split domain / application / infrastructure / http ; DI for adapters, the dependency rule points inward, cross-context communication via an asynchronous fire-and-forget in-process domain-event bus (@nestjs/cqrs), with a reconciliation sweep for dropped events. @nestjs/swagger derives openapi.json from the controllers and DTO classes — no schema DSL, no hand-written spec.	0002-ddd-hexagonal-backend
Streaming performance, one public origin	A dedicated nginx container serves the built SPA and reverse-proxies /api to the NestJS API — static files and Range streaming stay off the Node event loop; single origin keeps the session cookie first-party.	0003-nginx-serves-frontend
Portability, future multi-node	PostgreSQL for metadata + pluggable binary storage behind a FileStorage port (local filesystem default, S3-compatible optional).	0004-metadata-postgres-blob-storage-port
Per-user privacy	OIDC auth delegated to Keycloak (own container + keycloak-db); the backend is a confidential client with a backend-for-frontend session cookie (tokens held server-side); ownership enforced inside use cases (every repository query scoped by ownerId), with an AUTH_ENABLED=false mode backed by one implicit local user.	0005-session-cookie-auth
One-command operation	Everything is a Compose service; 12-factor env config; migrations run on API start; the S3 backend is a Compose profile , off by default.	—
Turntable UX	Frontend feature slice player/ with a single signal-based store as the source of playback truth;	—

	CSS animation gated on <code>prefers-reduced-motion</code> ; <code><audio></code> element wrapped by an <code>AudioService</code> .	
--	---	--

5 Building Block View

5.1 Level 1 — Whitebox Overall System

Building block	Responsibility
frontend/ — Angular SPA	Everything the browser runs: routes, the record-player UI, client-side playback state. No business rules — every mutation goes through the API. Detailed in Level 2 below.
backend/ — NestJS API	DDD/hexagonal contexts <code>songs</code> , <code>playlists</code> , <code>identity</code> , <code>streaming</code> (0002-ddd-hexagonal-backend); owns every invariant and every <code>ownerId</code> check. Its own building-block breakdown is <code>domain-model.typ</code> , not repeated here.
nginx (Compose web)	Serves the built SPA and reverse-proxies <code>/api/*</code> to the API (0003-nginx-serves-frontend); TLS termination.
PostgreSQL (Compose db)	Song & playlist metadata; local user rows.
Blob storage	Audio bytes behind <code>FileStorage</code> (0004-metadata-postgres-blob-storage-port): local filesystem by default, S3-compatible (MinIO in dev) behind a Compose profile.
Keycloak + keycloak-db	OIDC identity provider (0005-session-cookie-auth); the backend is a confidential client and holds tokens server-side.

The channel-level view (protocols, who calls whom) is in §3, Technical Context — not repeated here.

5.2 Level 2 — Frontend Whitebox

`frontend/src/app/` is Angular 22, zoneless, standalone components, organised by feature rather than by layer:

```
src/app/
  api/                generated HeyApi client + TanStack Query options (ADR-0006)
  api-runtime-config.ts  fetch client defaults (baseUrl /api, credentials include)
  auth/              AuthService (BFF redirects), auth-redirect (401 → login), UserMenuComponent
  core/              app-wide singletons (interceptors, guards) – currently empty
  shared/
    song-asset-urls.ts  cover/audio URL builders
    songs/              SongListPanelComponent (smart) + presentational row/menu/skeleton
  features/
    songs/             SongsPageComponent (route), SongUploadComponent
    playlists/         PlaylistsPageComponent, PlaylistDetailComponent (routes),
                        ui/ (PlaylistGridComponent, PlaylistCardComponent, ...)
    player/            PlayerService (signal store), PlayerComponent (bar),
                        PlayerDrawerComponent (expanded view), ui/ (transport,
                        scrubber, turntable, volume – all presentational)
  app.config.ts       providers: router, TanStack Query, i18n, Optimus UI theme
  app.routes.ts       /songs · /playlists · /playlists/:id
  e2e/               Playwright specs, run against the full Compose stack
```

State has exactly two homes, never a third:

- **Server state** — anything that came from the API (songs, playlists) — lives in TanStack Query, injected via `injectQuery/injectMutation` in whichever component owns that data (0006-openapi-typed-client-tanstack-query). Nothing duplicates it into a signal “just in case”.
- **Client state** — state that exists only in the browser — is either local to one component (`signal()/model()` — e.g. which song is open in the edit dialog) or, when several unrelated parts of the shell need it at once, lives in an injectable service (`PlayerService`, `providedIn: 'root'`, one `<audio>` element mirrored into signals; see §8).

Component roles follow a presentational/container split with a capped event-forwarding depth — the rule and its rationale are ADR-0007. The building blocks it produces:

Role	Examples
------	----------

Route component	SongsPageComponent, PlaylistsPageComponent, PlaylistDetailComponent — own a query, resolve route params, compose the panel/grid below them.
Smart panel	SongListPanelComponent — every song-row mutation (play, edit, delete, add/remove-from-playlist, reorder) for both the library and every playlist-detail view, so that logic exists exactly once.
Presentational — layout	SongListComponent, PlaylistGridComponent — lay out repeated children, re-emit their events unchanged, own no query or mutation.
Presentational — leaf	SongRowComponent, SongMenuComponent, PlaylistCardComponent, player/ui/* — take inputs, raise outputs, know nothing about HTTP.
Modal / form	SongEditDialogComponent, PlaylistCreateDialogComponent — Angular Signal Forms + their own mutation, opened by one input/output pair.

6 Runtime View

Two scenarios: one that crosses the whole stack, one that stays inside the frontend and traces how far a single event actually travels (ADR-0007).

6.1 Scenario: play a song from the library

Step	What happens
1	User clicks a row's play surface in <code>SongRowComponent</code> → <code>play.emit(song)</code> .
2	<code>SongListComponent</code> re-emits <code>play</code> unchanged (the one layout hop — the row is reused in both the plain and drag-reorderable layouts).
3	<code>SongListPanelComponent.play(song)</code> runs: the display order it was given (<code>rows()</code>) becomes the playback queue, then <code>PlayerService.play(song, queue)</code> .
4	<code>PlayerService</code> sets the <code><audio></code> element's <code>src</code> to <code>songAudioUrl(song.id) (/api/songs/:id/audio)</code> and calls <code>audio.play()</code> ; its <code>play/pause/timeupdate/loadedmetadata</code> listeners mirror the element's real state into signals.
5	The browser issues a range-capable GET for the audio file. nginx proxies it to the API with buffering off (0003-nginx-serves-frontend); the API streams the bytes from <code>FileStorage</code> (0004-metadata-postgres-blob-storage-port), honouring Range so 206 Partial Content responses let the browser seek without downloading the whole file.
6	Every consumer of <code>PlayerService</code> 's signals — the sticky bottom bar (<code>PlayerComponent</code>) and, if open, the expanded drawer (<code>PlayerDrawerComponent</code>) — re-renders from the same source of truth with no event passed between them; neither one owns or forwards playback state, both just inject the service.

Note what **doesn't** happen: no component between the row and the panel knows what “play” means, and nothing downstream of the panel is told to re-emit anything — the chain in step 1–3 is exactly the two hops ADR-0007 caps it at.

6.2 Scenario: delete a song from a playlist view

Traces the **deepest** event-forwarding chain in the app, end to end, and where it terminates.

1. `SongMenuComponent` (the ... menu) — user picks “Remove from Playlist” → `removeFromPlaylist.emit({ song, entryId })`.
2. `SongRowComponent` re-emits it unchanged (it owns the row's layout, not the menu's choices).
3. `SongListComponent` re-emits it unchanged again (it owns **which** rows are shown — plain list or drag list — not what a row's menu did).
4. `SongListPanelComponent.removeFromPlaylist(...)` — the chain ends here: a `confirm()`, then `playlistsControllerRemoveEntryMutation`, then both the playlist-detail query and the playlists-list query are invalidated (track counts change) so every screen showing this playlist refetches.

Three re-emitting hops, all unchanged pass-throughs, one owner. Compare to the **playback** scenario above, where step 6 needed no forwarding at all because the state that matters to more than one component was never an input/output chain to begin with — it was a service from the start.

6.3 Scenario: upload and parse a song

Step	What happens
1	User drags an <code>.mp3</code> onto <code>SongUploadComponent</code> , or picks one via the hidden file input; non- <code>.mp3</code> files are rejected client-side before any request is made.
2	<code>songsControllerUploadMutation</code> posts the file as <code>multipart/form-data</code> to <code>POST /api/songs</code> (session cookie, 0005-session-cookie-auth).
3	The API reads the ID3 tag (title/artist/album/duration), stores the audio via <code>FileStorage</code> , and persists the song row scoped to the caller's <code>ownerId</code> .
4	On success (or once the whole batch settles), the mutation invalidates the songs list query; <code>TanStack Query</code> refetches and every subscriber (the songs page, any playlist-detail panel currently open) re-renders with the new song — no manual cache surgery in the component.

7 Deployment View

8 Cross-cutting Concepts

8.1 State management (frontend)

Exactly two homes for state, as introduced in §5: **server state** in TanStack Query (0006-openapi-typed-client-tanstack-query), **client state** either local to a component or, when genuinely shared by several simultaneously mounted parts of the shell, in an injectable signal-based service (**PlayerService**). Nothing is duplicated between the two — a query result is never copied into a signal “for convenience”, and playback state is never persisted through the API.

8.2 Component communication (frontend)

Presentational leaves emit; layout components may re-emit an event unchanged **once**; a single smart component per concern owns the query/mutation or calls the shared service. Full rationale, alternatives considered, and the two-hop rule of thumb: ADR-0007. Traced examples: §6.

8.3 Authentication (frontend half)

Login/logout are full-page navigations to `/api/auth/*` (0005-session-cookie-auth) — the SPA never touches a token, only an opaque session cookie the browser sends automatically (`credentials: include`, same origin, 0003-nginx-serves-frontend). Two places react to being signed out: `auth-redirect.ts` installs a response interceptor on the generated client that sends the whole page to the BFF login on any 401; `UserMenuComponent` queries `/auth/me` to decide between showing the signed-in email or a Sign in / Create account pair. With `AUTH_ENABLED=false` (native dev) both branches stay dormant — the API never 401s, and `/auth/me` always resolves to the one implicit local user.

8.4 Internationalisation

@ngx-translate/core with one runtime-loaded locale (`public/i18n/en.json`); every user-facing string is a translation key, including strings used for Playwright/pally selectors — see §8.8 below.

8.5 Forms

Angular Signal Forms (@angular/forms/signals) — `form()`, `required`, `maxLength`, `submit()` — for the two mutating dialogs (song metadata, playlist creation). Validation messages are translated; `submit()` marks the form touched and no-ops on an invalid submit, so the mutation only ever fires with data that already passed client-side validation (the API still re-validates — this is UX, not the trust boundary).

8.6 Styling & theming

Tailwind CSS 4 utility classes plus @openng/optimus-ui component styles, one custom Aura theme preset (`app.config.ts`). Dark mode follows the OS preference; the turntable’s disc-spin and tone-arm animations are gated on `prefers-reduced-motion` (the top-priority quality goal, §1).

8.7 Accessibility

WCAG 2 AA is a CI-checkable gate, not a best-effort: `pnpm ally` (`pally-ci`, config `.pallyci.cjs`) runs an axe + HTML_CodeSniffer scan against the songs list, the playlists grid, and a playlist detail view on the live Compose stack, authenticating through the real Keycloak form via pally’s own action scripting (browser cookies aren’t reachable through pally’s plain `headers` option — see the comment at the top of `.pallyci.cjs` for why). `scripts/ally-session.mjs` seeds one song and one playlist over the REST API first so the scan sees populated list/detail markup, not empty states. Semantics leaned on throughout: native `<button>`s for every interactive control, `aria-label` on icon-only buttons (translated), and PrimeNG-derived components (`p-dialog`, `p-tieredmenu`) that already carry the relevant ARIA roles.

8.8 Testing strategy (frontend)

Layer	Tool / scope
Unit	Vitest (ng test, Angular 22’s default) — component and service logic in isolation.
Accessibility	pally-ci against the built app, see above.
End-to-end	Playwright (pnpm e2e), against the full Compose stack (<code>docker compose up -d</code> , <code>E2E_BASE_URL</code> defaults to <code>https://localhost:8443</code>) — real nginx, real NestJS API, real Postgres, real Keycloak. An <code>e2e/auth.setup.ts</code> project logs in once as the seeded dev user (<code>keycloak/import/wmp-realm.json</code>) through the actual hosted Keycloak form and saves the session as Playwright storage state, so the chromium project’s specs start already signed in. Specs share that one backend and one user with no per-test data isolation, so they run <code>fullyParallel: false</code> / <code>workers: 1</code> and each spec cleans up what it creates (<code>deleteSong</code> , deleting the playlist it made) — see §11.

Backend testing (unit / integration against a real Postgres via Testcontainers) is out of scope for this section; see `backend/README.md`.

9 Architecture Decisions

This document (arc42) is the **living** architecture description. Individual significant decisions are captured as standalone **ADRs** in `Documentation/adr/`, MADR-style: numbered, immutable once accepted, superseded rather than edited. Each ADR records context, the options considered, the decision, and its consequences. New decisions get the next number and a link back here.

ADR	Decision	Status
0001-monorepo-monolith	Single repository, single deployable monolith (<code>backend/</code> + <code>frontend/</code> + root <code>Compose</code>).	Accepted
0002-ddd-hexagonal-backend	DDD tactical patterns with hexagonal layering; one feature-folder module per bounded context; asynchronous fire-and-forget in-process domain-event bus.	Accepted
0003-nginx-serves-frontend	A dedicated nginx container serves the SPA and reverse-proxies <code>/api</code> , rather than serving static files from Express.	Accepted
0004-metadata-postgres-blob-storage-port	PostgreSQL for metadata; audio bytes behind a <code>FileStorage</code> port with local-FS default and optional S3 adapter.	Accepted
0005-session-cookie-auth	OIDC authentication delegated to Keycloak (own container + database); backend-for-frontend session cookie, tokens held server-side; ownership enforced inside use cases; <code>AUTH_ENABLED=false</code> local-user mode.	Accepted
0006-openapi-typed-client-tanstack-query	OpenAPI-first typed frontend client: <code>@nestjs/swagger</code> → <code>openapi.json</code> → <code>HeyApi</code> ; <code>TanStack Query</code> for server state.	Accepted
0007-frontend-component-communication	Presentational/container split; output events capped at two forwarding hops before a smart component; cross-cutting client state (playback) in an injectable signal service instead.	Accepted

10 Quality Requirements

10.1 Quality Tree

- **Usability**
 - Turntable metaphor is legible and responsive (Prio 1)
 - Clear loading / success / error feedback; designed empty states (NFR-1)
 - Keyboard operable, WCAG AA contrast, `prefers-reduced-motion` honoured
- **Performance efficiency**
 - Playback starts without full download; seek latency low on mobile (Prio 2)
 - Song list renders quickly with lazy-loaded cover thumbnails
- **Maintainability**
 - Domain logic unit-testable with no I/O (Prio 3)
 - Bounded contexts independently understandable; adapters swappable
- **Security**
 - Strict per-user data isolation (Prio 4)
 - Safe upload handling; hashed passwords; no secrets in the repo
- **Portability / Operability**
 - `docker compose up` on a clean machine, no manual steps (Prio 5)
 - Storage backend switchable to S3 by configuration only
- **Reliability**
 - Delete leaves no orphaned files or dangling playlist entries

10.2 Quality Scenarios

ID	Quality	Scenario (stimulus → response)	Measure
Q1	Performance	User drags the seek bar to a new position on a throttled mobile connection.	Audio resumes from the new position in < 1 s; a single 206 range request, no full-file fetch.
Q2	Security	User A requests <code>GET /api/songs/{B's id}</code> .	Always 404; covered by an integration test for every resource type.
Q3	Operability	<code>docker compose up</code> on a machine with only Docker installed.	App reachable on <code>:80</code> , migrations applied, no manual step, within 60 s.
Q4	Reliability / Security	Upload of a 25 MB file, or a <code>.wav</code> renamed to <code>.mp3</code> .	Rejected with 4xx; no meta-data row and no stored object created.
Q5	Maintainability	A new use case is added to the <code>playlists</code> module.	It can be fully unit-tested with fake ports; no PostgreSQL needed for that test.
Q6	Usability	OS “reduce motion” setting is on when a track plays.	The disc does not spin; controls and audio behave normally.
Q7	Reliability	A song that sits in 3 playlists and is currently playing is deleted.	Row + file removed, all 3 playlists lose the entry with order preserved, the player advances or stops.

11 Risks and Technical Debt

Area	Risk / debt	Mitigation / status
E2E & a11y test isolation	<code>pnpm e2e</code> and <code>pnpm a11y</code> both run against the real Compose stack with one seeded user and one shared Postgres — there is no per-test tenant or database reset. A spec that fails before its own cleanup step leaves orphaned rows (a song, a playlist) that later runs will see but ignore (they select by a fresh unique title, not by absence of prior data).	Specs run serially (<code>workers: 1</code>) so they don't race each other's uploads; each spec deletes what it created. Acceptable for a one-developer project; would need per-run tenant isolation (a fresh Keycloak user + DB schema per CI run) to scale to parallel CI.
Event-forwarding depth	The two-hop cap (ADR-0007) is a review convention, not an enforced lint rule — nothing stops a chain from quietly growing past it as new wrapper components are added.	No tooling yet. A custom ESLint rule (<code>max depth of (x)="x.emit(\$event)"</code> re-emission per event name) would catch it mechanically; not built — revisit if a real three-hop chain appears in review.
<code>.pallyci.cjs</code> re-authenticates per URL	Each of the 3 scanned URLs logs in through the real Keycloak form from scratch (<code>pally</code> has no cookie-jar option — see §8), adding a few seconds per URL and one more Keycloak session per run.	Acceptable at the current scan scope (3 URLs). Would need a shared-profile or CDP-cookie-injection approach to stay fast if the URL list grows significantly.
Frontend bundle budget	The production build already exceeds the default Angular initial-bundle budget (about 1.39 MB vs. a 1 MB warning threshold).	Not yet addressed — no route-level code-splitting beyond what the Angular CLI does by default. Low priority: local/self-hosted deployment, not a CDN-served public app (§3).

12 Glossary

Term	Definition
Aggregate / Aggregate root	A cluster of domain objects treated as one unit for data changes; the root is its only external entry point and guards its invariants.
Value Object	An immutable, equality-by-value domain type with no identity (e.g. <code>Email</code>).
Bounded Context	A boundary within which a domain model and its terms have one precise meaning; here <code>songs</code> , <code>playlists</code> , <code>identity</code> , <code>streaming</code> .
Port / Adapter	A port is an interface the domain/application defines; an adapter is a concrete implementation (DB, filesystem, HTTP).
Hexagonal architecture	Ports-and-adapters style: business logic in the centre, all I/O at the edges behind ports.
Composition root	The single place where the object graph is assembled and concrete adapters are chosen (<code>container.ts</code>).
Domain event	A record that something significant happened in the domain (e.g. <code>SongDeleted</code>), published on the in-process bus and handled by other contexts asynchronously, fire-and-forget.
ADR	Architecture Decision Record — a short, immutable document capturing one decision, its context and consequences.
arc42	A template for architecture documentation, 12 sections; this document.
Range request / 206	HTTP mechanism to fetch part of a resource (<code>Range: bytes=... → 206 Partial Content</code>); enables audio seeking.
OIDC	OpenID Connect — the identity layer on top of OAuth2 used to authenticate against Keycloak; the backend validates the realm-issued JWT access token per request.
ID3	Metadata tags embedded in MP3 files (title, artist, album, cover).
MoSCoW	Prioritisation scheme: Must / Should / Could / Won't.
MinIO	A self-hostable S3-compatible object storage server, used for local dev and tests.
SPA	Single-Page Application — the Angular frontend.